README :

Table des matières

Auteurs 1
Installation 1
Utilisation 2
ToDo 3
- Niveau de gris : 3
- Contraste : 3
- Assombrissement : 4
- Éclairage : 4
- Histogramme : 4

Algorithmes 4
Précisions 6

Auteurs

Julien NOEL et Rémi OLIVIERI du groupe 2

Installation

Ouvrez le terminal et, si ce n'est pas déjà le cas, allez vers votre dossier personnel en entrant la commande :

```
cd
```

Rentrez maintenant la commande :

```
gedit .bashrc
```

Soient P1, P2, P3, P4, P5 et P6 les chemins absolus menant, dans l'ordre, vers les fichiers respectifs :

jai_codec.jar
jai_core.jar
mlibwrapper_jai.jar
NOEL-OLIVIERI.jar
TestTraitementImage.java
libmlib_jai.so

Le fichier *.bashrc* étant ouvert, ajoutez-y les lignes suivantes :

```
export
CLASSPATH=$CLASSPATH:P1/jai_codec.jar:P2/jai_core.jar:P3/mlibwrapper_jai.jar:P4/N
OEL-OLIVIERI.jar:P5/TestTraitementImage.java
export LIBRARY_PATH=$LIBRARY_PATH:P6/libmlib_jai.so
```

Nous vous recommandons de mettre tous les fichiers JAR et le fichier SO dans le même dossier.

Sauvegardez et fermez le fichier *.bashrc* .

C'est à vous de choisir si vous voulez soit compiler et exécuter le fichier *TestTraitementImage.java*, soit exécuter directement *NOEL-OLIVIERI.jar*, **avec dans ce cas-ci l'obligation de mettre tous les fichier JAR dans le même dossier**, donc soit rentrer (dans un terminal) ces 3 lignes ci-dessous :

```
cd P5
javac -encoding UTF-8 TestTraitementImage.java
java TestTraitementImage
```

...soit rentrer cette ligne ci-dessous :

```
java -jar P4/NOEL-OLIVIERI.jar
```

Utilisation

Après avoir lancé le programme via une des deux méthodes citées ci-dessus, une fenêtre se lance. Voici la partie vraiment intéressante de cette fenêtre :



Il suffit de cliquer sur le bouton de gauche, de choisir l'image PNG (unique format de fichier accepté par le programme… faire attention car ce n'est pas parce que l'extension du fichier est *.png* que c'est forcément un fichier PNG…) que vous voulez traiter et de cliquer sur le bouton de droite.

Pour le reste, cela coule de source, explorez l'interface et appliquez le ou les effets souhaités.

Une documentation est disponible pour ce projet, il suffit de lancer le fichier *index.html* se trouvant dans *mam3/ipa/projet/html* après avoir décompresser le fichier *NOEL-OLIVIERI.jar* .

N.B. : La zone de texte du milieu accepte que l'on oublie de mettre l'extension *.png* au chemin absolu ou que l'on intervertisse slashs et antislashs et (sous Windows cette fois) que l'on oublie de mettre le disque où se trouve le fichier :

```
/home/user/Documents/image.png
/home/user/Documents/image
\home\user\Documents\image.png
```

Ces 3 lignes sont équivalentes pour le programme sous Linux.

```
C:/Documents/image.png
/Documents/image.png
```

Ces 2 lignes sont équivalentes pour le programme sous Windows.

ToDo

Ce programme, nommé *Image Processing* dans la documentation Doxygen, permet d'éditer une image PNG (via son chemin absolu).
Voici les effets disponibles :

- Niveau de gris (traitement) : transforme une image en couleurs en une image en niveau de gris, avec passage de 3 canaux à 1 canal de couleur uniquement, le nom de la nouvelle image se terminant par *-gr* ;
- Contraste (traitement) : transforme une image en négatif, le nom de la nouvelle image se terminant par *-co* ;

- Assombrissement (traitement) : assombri une image, le nom de la nouvelle image se terminant par *-da* ;
- Éclairage (traitement) : éclaircit une image, le nome de la nouvelle image se terminant par *-li* ;
- Histogramme (analyse) : génère un fichier TXT, dont le nom se terminera par *-h*, qui vous servira à faire une analyse de l'exposition via un graphique .

Tout fichier créé par ce programme se trouve dans le même dossier que celui de l'image choisie.

Tout processus générant une nouvelle image, sauf pour le niveau de gris, se sert d'une matrice de convolution, d'un *divisor* et d'un *offset* bien précis, processus se servant donc forcément du fichier *MatrixOperator.java* .

Avec plus de temps, nous aurions bien voulu traiter le canal de transparence α, ainsi que rajouter l'option où l'utilisateur rentre sa propre matrice de convolution, avec le *divisor* et le *offset* qu'il souhaite.

Tous les bugs survenus au cours du projet ont été réglés, il n'y en a donc dès à présent aucun !

Algorithmes

Pour le traitement « Niveau de gris » :

- On prend 3 coefficients entiers en paramètre. S'il y a une incohérence, on prendra des valeurs par défaut :
  - 30 pour le canal rouge
  - 59 pour le canal vert
  - 11 pour le canal bleu
- On crée un tableau d'entiers, chacun codés sur 8 bits, de même longueur que celui associé à l'image en couleurs.
- Pour chaque entier du tableau, on effectue la moyenne pondérée de chaque entier de chaque canal de couleur à l'aide des 3 coefficients.

- On crée une image en niveau de gris avec le nouveau tableau d'entiers.

Pour les traitements « Contraste », « Éclairage » et « Assombrissement » :

- On crée un fichier *convmatrix.csv* qui contient une matrice de convolution, que l'on note M, via la classe *MatrixOperator* (méthode statique *createMatrix()*).
- On applique cette matrice M à chaque pixel de chaque canal de couleur de l'image choisie via la classe *MatrixOperator* (méthode statique *convolution()*).
  Chaque entier résultant sera divisé par un entier *divisor*, puis on leur ajoutera un entier *offset*.

Pour le contraste : $M = \begin{pmatrix} 0 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 0 \end{pmatrix}$; $\text{divisor} = 1$; $\text{offset} = 255$

Pour l'assombrissement : $M = \begin{pmatrix} 0 & 0 & 0 \\ 0 & 4 & 0 \\ 0 & 0 & 0 \end{pmatrix}$; $\text{divisor} = 5$; $\text{offset} = 0$

(au lieu de mettre au carré et de normaliser)

Pour l'éclairage : $M = \begin{pmatrix} 0 & 0 & 0 \\ 0 & 5 & 0 \\ 0 & 0 & 0 \end{pmatrix}$; $\text{divisor} = 4$; $\text{offset} = 0$

(au lieu de prendre la racine carré et de normaliser)

Pour l'analyse « Histogramme » :

- On crée un fichier *N-h.txt* vide, où *N* est le nom de l'image choisie.
- On remplit le tableau d'entiers *matConv[]* avec uniquement des zéros.
- *matConv[]* stockera les futures lignes (les unes après les autres et dans le même ordre) du fichier texte.
- La première colonne du fichier texte contiendra toutes les valeurs entières comprise entre 0 et 255 inclus.
- Chaque autre colonne sera associée à un canal de couleur et contiendra, sur chaque ligne, le nombre de pixels dont la valeur associée au canal vaut l'entier correspondant de la première colonne.

Par rapport à la classe *MatrixOperator* (et donc le fichier *MatrixOperator.java)* :

- Pour la fonction statique *createMatrix()* :
  - On crée un fichier *convmatrix*.csv avec les paramètres rentrés par l'utilisateur s'ils sont cohérents, sinon rien ne se passe.

- Pour la fonction statique *convolution()* :
    - On charge la matrice contenu dans le fichier *convmatrix.csv* lorsque cela est possible et on la stocke dans le tableau à une dimension *matConv[]*, ce dernier ne contient que des zéros si jamais un problème est rencontré au cours du processus (si, par exemple, le fichier CSV contenait une matrice non carrée).
    - À **chaque** pixel du tableau donné en paramètre de la fonction, on applique la matrice de convolution stockée dans *matConv[]* en procédant ainsi :
        - On (ré)initialise l'entier *sum* à zéro (on a *i* un entier compris entre 0 et la taille de *matConv[]* moins 1).
        - On vérifie qu'à chaque coefficient a_i de *matConv[]* est associé un entier b_i qui appartient au tableau et qui se situe sur la bonne ligne lorsque l'on interprète le tableau comme étant une matrice stockant les lignes les unes à la suite des autres. Sinon on pose $b_i = 0$. Cette vérification est en grande partie effectuée par la fonction *indexForMultiplication()*.
        - sum $= \sum_i a_i \times b_i$
        - On divise *sum* par *divisor* puis on ajoute *offset* et on stocke le résultat dans un nouveau tableau d'entier étant de la même taille que le tableau en paramètre.
        - On effectue ce procédé autant de fois qu'il y a d'entiers dans le tableau en paramètre (et donc qu'il y a de pixels contenus dans l'image à traiter).
    - On renvoie ce nouveau tableau créé.

Précisions

- La documentation créée à l'aide de Doxygen ne prend pas en compte tous les attributs et méthodes privés.
- Toutes les méthodes contenues dans l'ensemble des fichiers JAVA servent ou ont servi à un moment donné au cours du projet.
- Si le programme ne marche pas comme souhaité après l'exécution de *NOEL-OLIVIERI.jar*, (re)lisez bien la rubrique « Utilisation ».😉
- Nous vous remercions d'avoir lu jusqu'ici !